

DOCKER CAPSTONE · REAL CLIENT CASE

SkillSpace

จากโจทย์ธุรกิจ 3 สาขา

สู่ระบบ **web · api · db** ที่ส่งมอบได้ด้วยคำสั่งเดียว

2 HANDS-ON LABS

15 EXPERIMENTS

3 CONTAINERS

01

โจทย์จากลูกค้า

ก่อนจะมีระบบ — ลูกค้าคือใคร วันนี้เขาทำงานกันอย่างไร และประโยคไหนในบทสัมภาษณ์ที่กลายเป็นข้อกำหนดในภายหลัง

SkillSpace — พื้นที่ฝึกอบรมที่ต้องพร้อมใช้ทุกวัน

บริษัทให้บริการศูนย์ฝึกอบรม ห้องประชุม และห้องคอมพิวเตอร์แบบพร้อมใช้สำหรับองค์กร



ธุรกิจของ SkillSpace

ลูกค้าเช่าพื้นที่เป็นรายชั่วโมงหรือรายวัน เพื่อจัดอบรม ประชุม และเวิร์กช็อป โดย SkillSpace ดูแลสถานที่ อุปกรณ์ และทีมสนับสนุนพนักงาน

ลูกค้าหลัก

ฝ่ายทรัพยากรบุคคล บริษัทฝึกอบรม วิทยากร และทีมงานองค์กรที่ต้องการห้องพร้อมใช้งานทันที

เมื่อห้องไม่พร้อม

การอบรมล่าช้า ลูกค้าไม่พอใจ และบริษัทเสียรายได้ — งานซ่อมและครุภัณฑ์จึงเป็นส่วนหนึ่งของธุรกิจ ไม่ใช่งานหลังบ้านที่รอได้

ทุกห้องที่พร้อมใช้ คือ ประสบการณ์ของลูกค้าและรายได้ของบริษัท



ห้องอบรมและห้องประชุมต้องพร้อมก่อนลูกค้าเข้าพื้นที่



ทีมงานตรวจสอบอุปกรณ์ เครื่องฉาย และครุภัณฑ์ก่อนเริ่มคลาส

24

ห้องอบรมและห้องประชุม

6

ห้องคอมพิวเตอร์

180

ครุภัณฑ์หมุนเวียนให้ยืม

บริษัท SkillSpace Services จำกัด — ศูนย์ฝึกอบรม 3 สาขา

เริ่มจากองค์กร คน และวิธีทำงานปัจจุบัน ก่อนฟังปัญหาจริงจากแต่ละบทบาทในสองหน้าถัดไป

3

สาขาที่ต้องดูแล

24

ห้องอบรมและห้องประชุม

6

ห้องคอมพิวเตอร์

180

ครุภัณฑ์หมุนเวียนให้ยืม (ชิ้น)

ตำแหน่ง	หน้าที่	จำนวน
ผู้จัดการฝ่ายปฏิบัติการ	รับเรื่อง จัดลำดับ มอบหมายงาน และรายงานผู้บริหาร	1 คน
ช่างซ่อมบำรุง	ตรวจห้องและอุปกรณ์ ซ่อมหน้างาน และเบิกอะไหล่จากคลัง	3 คน
เจ้าหน้าที่ครุภัณฑ์	ดูแลการยืม-คืนอุปกรณ์ และควบคุมสต็อกอะไหล่	1 คน

ทีมปฏิบัติการหลักมี **5 คน** และใช้ระบบจากศูนย์ควบคุมเดียวกัน — เวอร์ชันสอนนี้จึงยังไม่ทำระบบล็อกอิน

วันนี้เขาทำงานกันอย่างไร

แจ้งปัญหา

:

วิทยากรหรือพนักงานส่งข้อความ → ผู้จัดการจัดใส่สมุด → โทรบอกช่าง

ยืมของ

:

เดินมาที่ศูนย์ปฏิบัติการ → เขียนชื่อในสมุดยืม-คืน → หยิบของไป

เบิกอะไหล่

:

ช่างหยิบจากตู้เอง → บอกเจ้าหน้าที่ครุภัณฑ์ที่หลัง (บางครั้งก็ลืมบอก)

รายงาน

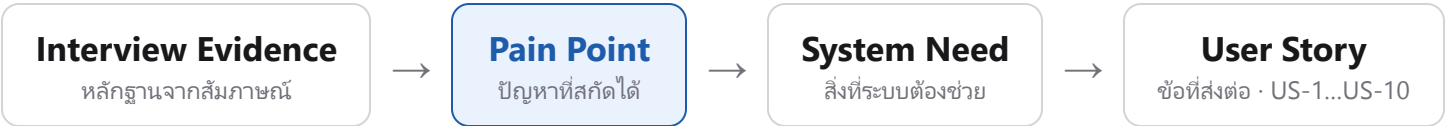
:

สิ้นเดือนนับนับจากสมุดที่ละหน้า

จากคำพูดของผู้จัดการ → User Stories ที่ต้องมี

สถานการณ์งานที่กำลังศึกษา: รับแจ้งปัญหา → มอบหมายช่าง → ติดตามงานค้างและรายงานผล

INTERVIEW EVIDENCE หลักฐานจากคำพูดผู้จัดการ	PAIN POINT ปัญหาที่สกัดได้	SYSTEM NEED สิ่งที่ระบบต้องช่วย	USER STORY ข้อที่ส่งต่อ
“ข้อความเลื่อนขึ้นไปแล้วผมลืมจด... เรื่องหายและตอบไม่ได้”	ไม่มีรายการกลางให้ตามงานจนจบ	สร้างใบแจ้งปัญหาและเห็นสถานะ	US-1 · เรื่องไม่หาย
“ไม่รู้ว่างานไหนค้างกี่วัน... ต้องนั่งนับเอง ครึ่งวัน”	ไม่มีจำนวนตามสถานะและงานค้าง	สรุปสถานะและชี้งานเกินกำหนด	US-6 · ภาพรวมงาน
“อยากเห็นหน้าจอดีว่า งานอยู่ในมือใคร”	มองไม่เห็นเจ้าของและความคืบหน้า	มอบหมายและติดตามสถานะ	US-2 · US-4 · เจ้าของงาน
“งานด่วนปนกับงานทั่วไป จนบางครั้ง เห็นซ้ำเกินไป”	ลำดับความสำคัญไม่เด่นชัด	แสดงความเร่งด่วนและงานที่ต้องเร่ง	US-6 · เห็นงานสำคัญ
“ตอนปิดงาน อยากรู้ว่า ซ่อมอะไรและใช้อะไหล่อะไร”	ผลการซ่อมกับการใช้อะไหล่แยกกัน	ปิดงานพร้อมผลซ่อมและอะไหล่	US-5 · US-10 · ตรวจสอบย้อนหลัง

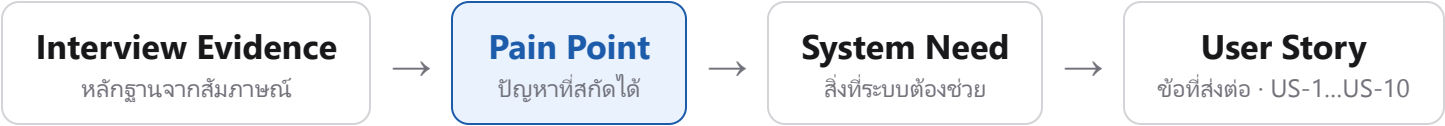


กติกา: ทุก User Story และ Functional Requirement ต้องย้อนกลับมาหาหลักฐานในตารางนี้ได้ มิฉะนั้นต้องตัดออกหรือกลับไปถามเพิ่ม

จากคำพูดของทีมงาน → User Stories ที่ต้องมี

สถานการณ์งานที่กำลังศึกษา: ช่างรับและปิดงาน → ยืม-คืนครุภัณฑ์ → เบิกและตรวจสอบตู้เอกสาร

ROLE บทบาท	INTERVIEW EVIDENCE หลักฐานจากคำพูดทีมงาน	PAIN POINT ปัญหาที่สกัดได้	SYSTEM NEED สิ่งที่ระบบต้องช่วย	USER STORY ข้อที่ส่งต่อ
ช่างซ่อมบำรุง	"อยากรู้ว่า งานไหนเป็นของผม"	ไม่เห็นงานในความรับผิดชอบ	กรองงานตามช่าง	US-3 · งานของฉัน
ช่างซ่อมบำรุง	"ก่อนเดินไป อยากรู้เห็น ห้อง อุปกรณ์ และอาคาร"	ข้อมูลหน้างานมาไม่ครบ	แสดงรายละเอียดใบงาน	US-3 · เตรียมงาน
ช่างซ่อมบำรุง	"อยาก บันทึกรายงานตอนปิดงาน"	ยอดอะไหล่ไม่สัมพันธ์กับงานซ่อม	ปิดงานพร้อมบันทึกอะไหล่	US-5 · US-10
เจ้าหน้าที่ครุภัณฑ์	"ของชิ้นเดียวถูกจอง วันเดียวกันสองคน"	สมุดป้องกันยืมซ้ำไม่ได้	ปฏิเสธของที่ยังไม่คืน	US-7 · กันยืมซ้ำ
เจ้าหน้าที่ครุภัณฑ์	"ของที่กำลังซ่อม ไม่ควรถูกยืมออกไป"	สถานะซ่อมไม่เชื่อมการยืม	ปฏิเสธของที่กำลังซ่อม	US-8 · ของต้องพร้อม
เจ้าหน้าที่ครุภัณฑ์	"อะไหล่หมดโดยไม่รู้ตัว"	ไม่เห็นยอดและจุดสั่งซื้อ	เตือนเมื่อใกล้หมด	US-9 · สต็อกต่ำ
เจ้าหน้าที่ครุภัณฑ์	"ยอดเปลี่ยนแล้วอยากรู้ว่า รับเข้าหรือเบิกกับงานไหน"	ตรวจสอบเหตุยอดเปลี่ยนไม่ได้	เก็บประวัติความเคลื่อนไหว	US-10 · ตรวจสอบย้อนหลัง



รวมสองหน้าได้ US-1...US-10: ใช้คำว่า User Story เป็นแกนเดียวตลอดชุด และทุกข้อย้อนกลับไปหาคำพูดจากบทสัมภาษณ์ได้

เสียงจากหน้า 6-7 → User Story ทั้ง 10 ข้อ

หนึ่ง User Story สรุปให้ครบว่า ใคร อยากทำ อะไร เพื่อให้ได้ คุณค่าอะไร และต้องย้อนกลับไปหา Interview Evidence ได้

งานแจ้งซ่อม · US-1...US-5		
US	จากหน้า	ใคร → อยากทำอะไร → เพื่ออะไร
US-1	6	ผู้แจ้ง → แจ้งห้อง อุปกรณ์ อาคาร → เรื่องไม่หาย
US-2	6	ผู้จัดการ → มอบหมายช่าง → รู้ว่างานอยู่ที่ใคร
US-3	7	ช่าง → ดูงานของตน → เริ่มงานโดยไม่ต้องถาม
US-4	6	ช่าง → อัปเดตสถานะ → ทุกคนเห็นความคืบหน้าเดียวกัน
US-5	6-7	ช่าง → ปิดงานพร้อมอะไร → ยอดคงเหลือตรง

งานควบคุม · US-6...US-10		
US	จากหน้า	ใคร → อยากทำอะไร → เพื่ออะไร
US-6	6	ผู้จัดการ → ดูสถานะและงานค้าง → รายงานได้ทันที
US-7	7	เจ้าหน้าที่ → กันยืมเข้า → ไม่จ่ายของชิ้นเดียวให้สองคน
US-8	7	เจ้าหน้าที่ → กันยืมของที่ซ่อม → ส่งมอบของที่พร้อม
US-9	7	เจ้าหน้าที่ → เห็นสต็อกต่ำ → สั่งซื้อก่อนงานสะดุด
US-10	6-7	เจ้าหน้าที่ → ดู stock movement → ตรวจที่มายอดได้



จำนวนไม่จำเป็นต้องเท่ากัน: หลักฐานหลายคำพูดอาจสนับสนุน User Story เดียวกัน และ User Story หนึ่งข้ออาจย้อนกลับไปได้มากกว่าหนึ่งหน้า

Interview Evidence → Functional หรือ Non-Functional?

อย่ารีบตั้งชื่อทุกคำพูดว่า User Story ให้ถามก่อนว่าเป็น "เป้าหมายของผู้ใช้" หรือ "ความเสี่ยงและข้อจำกัดของธุรกิจ"

จุดเริ่มร่วมกัน · Interview Evidence
คำพูด สิ่งที่เกิดขึ้น เอกสาร วิธีทำงานเดิม และเหตุการณ์ที่ลูกค้าบอกว่ารับไม่ได้ — ทั้งหมดคือหลักฐานจาก Discovery หน้า 5-7

สายที่ 1 · เป้าหมายของผู้ใช้ → Functional
"เรื่องแจ้งซ่อมต้องไม่หาย" → สรุปเป็น **US-1** ผู้ใช้ต้องการแจ้งปัญหา → แปลงเป็น **FR-01** ระบบสร้าง ticket ได้

Evidence
เป้าหมายผู้ใช้

→

User Story
ใคร · ทำอะไร · เพื่ออะไร

→

FR
ระบบต้องทำอะไร

สายที่ 2 · ความเสี่ยง/ข้อจำกัด → Non-Functional
"สร้าง Container ใหม่แล้วข้อมูลหาย" → สรุปเป็น **Business Risk** → แปลงเป็น **NFR-2** ข้อมูลต้องอยู่ยาวกว่า Container

Evidence
เหตุการณ์ที่รับไม่ได้

→

Risk / Constraint
คุณภาพที่ต้องรักษา

→

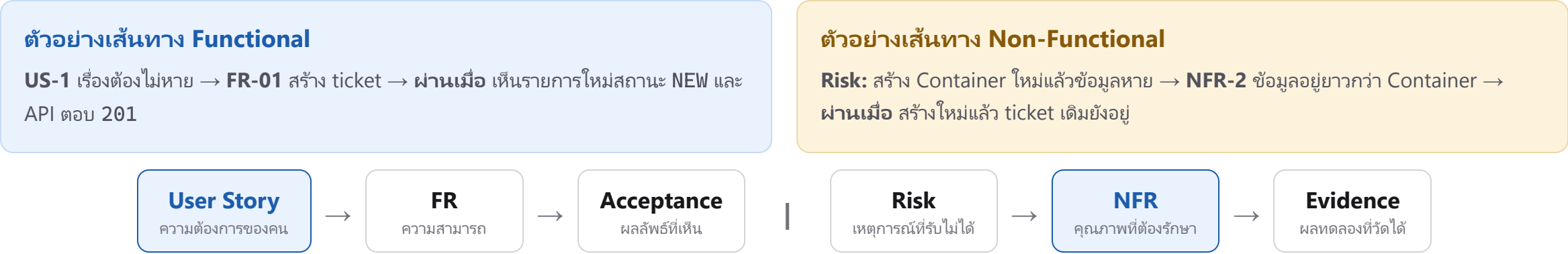
NFR
เงื่อนไขที่วัดได้

กติกาตัดสิน: ถ้าอธิบายคุณค่าที่คนต้องได้รับ ให้เขียน User Story; ถ้าอธิบายคุณภาพหรือเหตุการณ์ที่ธุรกิจรับไม่ได้ ให้เขียน Risk/Constraint โดยไม่ต้องฝืนสร้าง User Story

User Story → FR · Business Risk → NFR

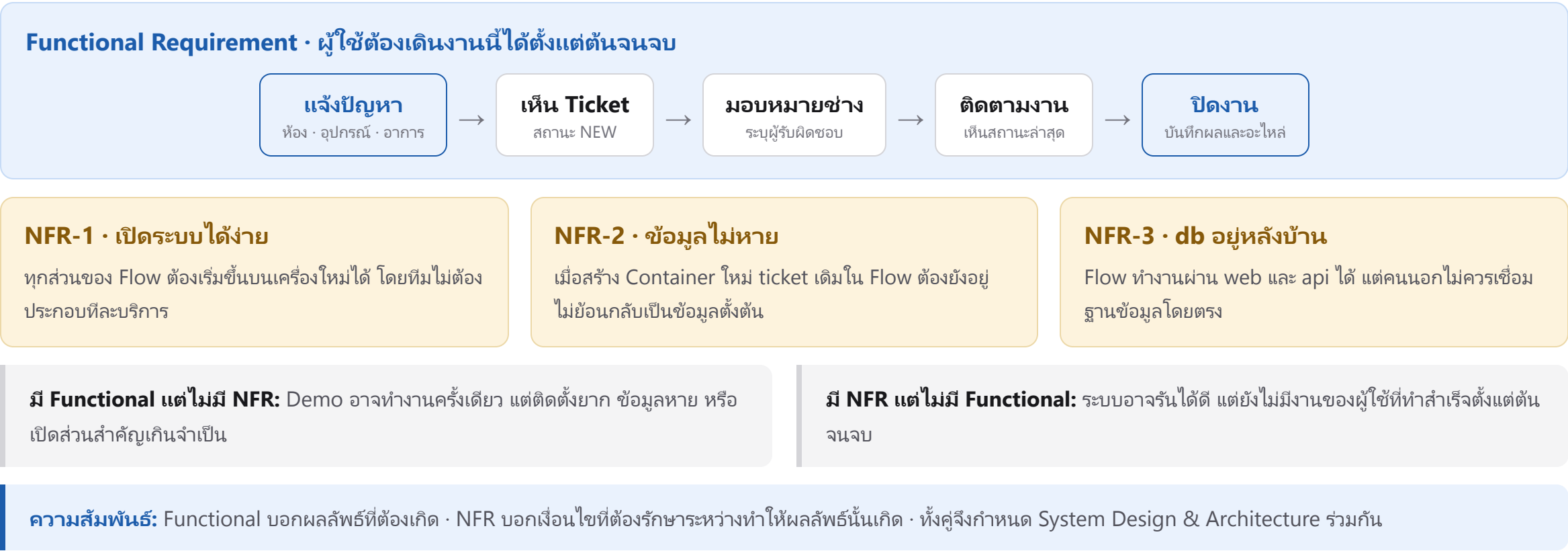
ข้อกำหนดทั้งสองแบบมีต้นทางและวิธีพิสูจน์ต่างกัน จึงไม่ควรเรียกทุกอย่างว่า User Story

ประเภท	ต้นทาง	คำถามของข้อกำหนด	เงื่อนไขการผ่าน
Functional Requirement (FR)	User Story บทบาทต้องการผลลัพธ์อะไร	ระบบต้องมีความสามารถหรือพฤติกรรมอะไร	Acceptance Criteria: ผู้ใช้ทำงานสำเร็จและเห็นผลลัพธ์ที่ระบุไว้
Non-Functional Requirement (NFR)	Business Risk / Constraint ธุรกิจรับเหตุการณ์ใดไม่ได้	ระบบต้องรักษาคุณภาพหรือเงื่อนไขอะไรระหว่างทำงาน	Measurable Evidence: ทดลองภายใต้เงื่อนไขที่กำหนดแล้วคุณภาพนั้นยังคงอยู่



Functional คือ เส้นทางงาน · NFR คือ เงื่อนไขที่ครอบคลุมทั้งเส้นทาง

สองอย่างไม่ต่อแถมกัน แต่ใช้พร้อมกันเพื่อให้ระบบ "ทำงานได้" และ "ส่งมอบใช้จริงได้"



หนึ่งแถวในตาราง เล่าเรื่องอะไรบ้าง?

อย่าเริ่มจากการจำรหัส — ให้ตามเส้นทางจาก “ความต้องการอะไร” ไปจนถึง “ทีมยอมรับงานได้เมื่อไร”



ตัวอย่าง · อ่านจากซ้ายไปขวา

US-1
ผู้ใช้ต้องการแจ้งปัญหาให้ทีมเห็นทันที

→

FR-01
FR = Functional Requirement
ระบบสร้างใบแจ้งปัญหาได้

→

Acceptance Criteria
ส่งแล้วเห็นรายการใหม่สถานะ NEW

คำว่า “ถือว่าผ่านเมื่อ...” คือคำแปลแบบภาษาคนของ **Acceptance Criteria** หรือ “เกณฑ์ยอมรับงาน”

ไม่ใช่ **Test Case** ทั้งหมด — มันบอกเส้นชัยที่ชัด ก่อนที่ทีมจะเขียนขั้นตอนทดสอบอย่างละเอียด

จากการแจ้งปัญหา → จัดการงานซ่อมได้จริง

FR = Functional Requirement · FR-01...FR-06 แปลงเรื่องของผู้ใช้ให้เป็นพฤติกรรมของระบบ และผูกทุกข้อกับเกณฑ์ยอมรับงานที่ตรวจได้

รหัส	FUNCTIONAL REQUIREMENT · ระบบต้องทำอะไร	ACCEPTANCE CRITERIA · เกณฑ์ยอมรับงาน	ต้นทาง
FR-01	สร้างใบแจ้งปัญหาโดยเลือกครุภัณฑ์ (ระบบดึงห้อง/ตำแหน่งจาก <code>assets.location</code>) และระบุหัวข้ออาการ/รายละเอียด ความเร่งด่วน	ข้อมูลครบ → API ตอบ 201 และเห็นการ์ดใหม่สถานะ <code>NEW</code> ; ข้อมูลไม่ครบ → API ตอบ 422 และไม่สร้างการ์ด	US-1
FR-02	งานเปลี่ยนสถานะตามลำดับเท่านั้น	ระบบไม่ยอมให้ข้ามจาก <code>NEW</code> ไป <code>DONE</code> และสถานะเดิมยังอยู่	US-4
FR-03	มอบหมายงานโดยระบุและแสดงชื่อช่างผู้รับผิดชอบ	ไม่ระบุชื่อ → ไม่เปลี่ยนเป็น <code>ASSIGNED</code> ; สำเร็จ → การ์ดเดิมแสดงชื่อช่างและสถานะ <code>ASSIGNED</code>	US-2
FR-04	กรองรายการตามช่างผู้รับผิดชอบได้	ช่างเปิดดูแล้วเห็นเฉพาะงานที่มอบหมายให้ตน	US-3
FR-05	ปิดงานพร้อมบันทึกอะไรก็ได้	งานเป็น <code>DONE</code> และยอดอะไรก็ได้ลดลงตามจำนวนที่บันทึก	US-5
FR-06	เบิกอะไรก็ได้มากกว่ายอดที่มีไม่ได้	ระบบปฏิเสธรายการ และยอดคงเหลือไม่เปลี่ยน	US-5

อ่านหนึ่งแถวแบบนี้: User Story ต้นทาง → ความสามารถที่ต้องสร้าง → หลักฐานที่ทำให้ Product Owner พุดได้ว่า “รับงานข้อนี้”

จากข้อมูลรายวัน → ตัดสินใจและป้องกันความผิดพลาด

FR = Functional Requirement · FR-07...FR-12 ครอบคลุมประวัติ รายงาน ยืม-คืน และสต็อก โดยใช้แพตเทิร์น FR + Acceptance Criteria เดียวกัน

รหัส	FUNCTIONAL REQUIREMENT · ระบบต้องทำอะไร	ACCEPTANCE CRITERIA · เกณฑ์ยอมรับงาน	ต้นทาง
FR-07	เก็บประวัติรับเข้าและเบิกอะไหล่	เปิดดูแล้วเห็นรายการเคลื่อนไหวเรียงตามเวลา	US-10
FR-08	สรุปจำนวนงานแยกตามสถานะ	ตัวเลขทั้ง 4 สถานะตรงกับรายการงานจริง	US-6
FR-09	ชิ้นงานที่ค้างเกินกำหนด	งานที่เกินเวลาตามระดับความเร่งด่วนปรากฏในรายการเตือน	US-6
FR-10	ห้ามยืมครุภัณฑ์ที่ยังไม่ถูกคืน	ระบบปฏิเสธทันทีเมื่อชิ้นนั้นมีรายการยืมค้างอยู่	US-7
FR-11	ห้ามยืมครุภัณฑ์ที่กำลังซ่อม	ระบบปฏิเสธเมื่ออุปกรณ์ยังมียานซ่อมที่ไม่เสร็จ	US-8
FR-12	เตือนเมื่ออะไหล่ต่ำกว่าจุดสั่งซื้อ	รายการที่ <code>qty_on_hand < reorder_point</code> ถูกทำเครื่องหมายในหน้า Parts และรวมในรายการเตือนหน้า Overview	US-9

สรุป: FR-01...FR-12 คือ Functional Requirement ทั้งหมด เพราะบอกความสามารถของระบบ · ส่วน NFR หน้าถัดไปจะบอกว่า Flow เหล่านี้ต้องทำงาน “ดีและไวใจได้” อย่างไร

กลัวอะไร → รับปากอะไร → ยอมรับงานเมื่อไร

ทั้งสามข้อเริ่มจากเหตุการณ์ที่ธุรกิจรับไม่ได้ แล้วแปลงเป็น NFR, Acceptance Criteria และ Evidence ที่ตรวจซ้ำได้

รหัส	เหตุการณ์ที่กลัว	NFR · เงื่อนไขที่รับปาก	ACCEPTANCE CRITERIA · เกณฑ์ยอมรับงาน	EVIDENCE
NFR-1	เครื่องหลักเสียก่อนเริ่มคลาส	เปิดระบบบนเครื่องใหม่ได้ง่าย	ใช้คำสั่งเดียวแล้ว web, api และ db ขึ้นครบพร้อมทำงาน	LAB 002
NFR-2	สร้าง Container ใหม่แล้วข้อมูลหาย	ข้อมูลมีอายุยาวกว่า Container	ลบแล้วสร้าง Container ใหม่ ticket ที่เพิ่มไว้ยังอยู่ครบ	LAB 001 · LAB 002
NFR-3	คนนอกต่อฐานข้อมูลโดยตรง	db อยู่ใน network ภายใน	web ใช้งานได้ครบ แต่ไม่มีพอร์ต db ถูกเปิดออกมาที่เครื่องผู้ใช้	LAB 002



แพตเทิร์นเดียวกับ Functional: ทุก Requirement ต้องบอกเส้นชัยที่ชัด · ต่างกันเพียง Functional วัด “ความสามารถ” ส่วน NFR วัด “คุณภาพและเงื่อนไข” · เมื่อสองชุดนี้พร้อม จึงนำไปออกแบบ User Flow, Architecture และ System Design ในตอนถัดไป

FR + NFR พร้อมแล้ว — ใช้ต่ออย่างไร?

FR บอกงานที่ผู้ใช้ต้องทำสำเร็จ ส่วน NFR เป็นเงื่อนไขที่ทุกชั้นและทุกส่วนประกอบต้องรักษา



หัวข้อถัดไป	รับข้อมูลจากอะไร	นำ REQUIREMENT ไปตัดสินใจเรื่องใด	ผลลัพธ์ที่ต้องได้
User Flow	FR และ Acceptance Criteria	เรียงขั้นตอน หน้าจอ การกระทำ สถานะสำเร็จ และกรณีผิดพลาด	ลำดับที่ผู้ใช้เดินจนได้ผลลัพธ์
Architecture	User Flow + NFR	เลือก component, connection, trust boundary และที่เก็บข้อมูลให้รองรับ Flow	ภาพ browser → web → api → db
System Design	FR + NFR + Architecture	ลงรายละเอียด UI, API contract, state rule, validation, schema และ persistence	ข้อกำหนดทางเทคนิคที่พร้อมสร้าง

ตัวอย่างเดี่ยว · งานแจ้งซ่อม			
Requirement	User Flow	Architecture	System Design
FR-01 สร้าง ticket NFR-2 ข้อมูลไม่หาย NFR-3 db อยู่ภายใน	กรอกปัญหา → กดส่ง → เห็น ticket สถานะ NEW	browser → web → api → db db ไม่เปิดพอร์ตและมี volume	ฟอร์มที่ต้องกรอก · POST /api/tickets · status enum · tickets table · volume mapping

วงจรมัด: Acceptance Criteria และ Evidence จากหน้า 13–15 จะย้อนกลับมาใช้ตรวจว่า User Flow และ System Design ที่สร้างขึ้นตอบ Requirement จริง ไม่ใช่เพียงวาดภาพสวย

03

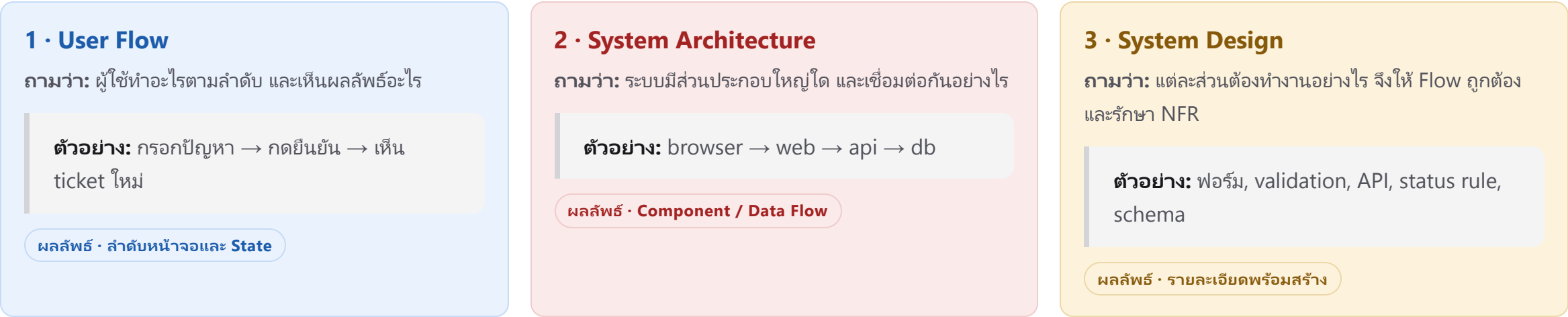
User Flow → Architecture → System Design

เริ่มจาก **User Flow** และสิ่งที่คนเห็นบนหน้าจอ · ต่อด้วย **System Architecture** ว่าส่วนประกอบใดร่วมกันทำงาน ·

แล้วจึงลง **System Design** ที่กติกา API, State และข้อมูล

User Flow · Architecture · System Design — ต่างกันที่คำถาม

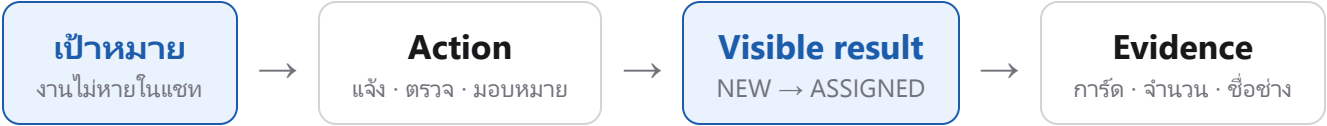
ทั้งสามมองระบบเดียวกัน แต่ชุมชนละระดับ ตั้งแต่สิ่งที่ผู้ใช้ทำ ไปถึงรายละเอียดที่นักพัฒนาต้องตัดสินใจ



ตัวอย่าง: แจ้งซ่อมจนมอบหมาย — ผู้ใช้เห็นอะไรที่หน้าจอ

State = สถานะที่ระบบเก็บให้ ticket ใบเดิม: NEW → ASSIGNED → IN_PROGRESS → DONE · **Visible State** = คอลัมน์ จำนวน และชื่อช่างบน UI ที่สะท้อน State นั้น

หน้าจอ	สิ่งที่ผู้ใช้เห็น	สิ่งที่ผู้ใช้ทำและระบบตอบสนอง
1 · Overview	จำนวนงาน 4 State · งานค้างเกินกำหนด · งานในมือช่าง · อะไหล่ที่ต้องสั่ง	ผู้ใช้อ่านค่าเริ่มต้นแล้วกด “กระดานงานซ่อม”; ระบบเปิดข้อมูลล่าสุด ไม่ใช่ค่าที่ฝังตายตัว
2 · Tickets	ฟอร์ม “แจ้งซ่อมใหม่” อยู่เหนือกระดาน NEW → ASSIGNED → IN_PROGRESS → DONE	ผู้ใช้เริ่มจากฟอร์มและเห็นปลายทางของงานพร้อมกัน; งานจะย้ายได้ทีละขั้นเท่านั้น
3 · New ticket form	รายการครุภัณฑ์จริง · หัวข้อ · รายละเอียด · ความเร่งด่วน พร้อมค่าที่เลือกไว้	ผู้ใช้กรอกให้ครบแล้วตรวจทาน; ถ้าหัวข้อว่างหรือค่าไม่ถูกต้อง ระบบไม่สร้างงานและแสดงข้อผิดพลาด
4 · Submit result	สำเร็จ: การ์ดใหม่ใน “รอรับเรื่อง” · ไม่สำเร็จ: ข้อความอธิบายโดยไม่มีการ์ดปลอม	กด “แจ้งซ่อม”; เมื่อสำเร็จ State เริ่มที่ NEW และจำนวนเพิ่ม 1 เมื่อไม่ผ่านให้แก้ไขข้อมูลเดิมแล้วส่งใหม่
5 · Assignment result	ช่องชื่อช่างบนการ์ด NEW; หลังสำเร็จการ์ดเดิมอยู่ “มอบหมายแล้ว” พร้อมชื่อช่าง	กรอกชื่อแล้วกด “มอบหมาย”; ถ้าไม่กรอก ระบบแจ้งว่าต้องระบุผู้รับผิดชอบและไม่เปลี่ยน State
6 · Overview (กลับ UI 1)	ก่อน: งานยังไม่ปิด 6 · มอบหมายแล้ว 2 · ยังไม่มี TECH-04 หลัง Flow: 7 · 3 · TECH-04 = 1	กลับ Overview แล้วเทียบก่อน/หลัง; ticket id เดิม ชื่อช่าง State ASSIGNED และจำนวนที่เปลี่ยนคือ end-to-end evidence

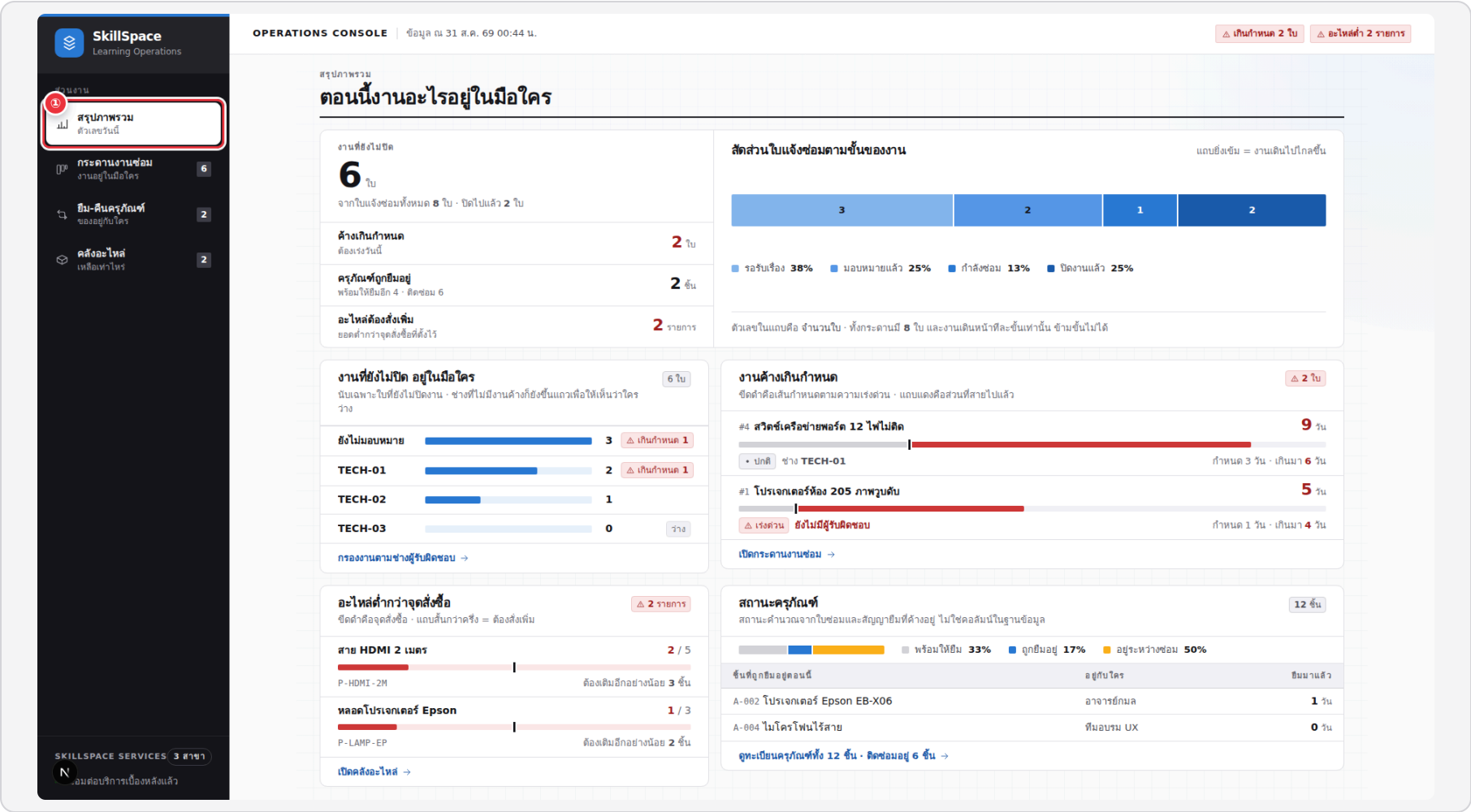


Traceability รายหน้าจอ: Overview: US-6 → FR-08/09 · US-9 → FR-12 (งานในมือช่างอาศัย FR-03; เปิดรายการเฉพาะช่างอาศัย FR-04) · Tickets/Form/Submit: US-1 → FR-01 · US-4 → FR-02 · Assignment: US-2 → FR-03 · US-4 → FR-02 · Overview verification: US-6 → FR-08

Cross-cutting NFR — พิสูจน์แยกจากภาพ UI: NFR-1 เปิดระบบด้วยคำสั่งเดียว · NFR-2 ข้อมูลยังอยู่หลังสร้าง Container ใหม่ · NFR-3 db ไม่เปิดพอร์ตออกภายนอก

หน้าจอแรก: Overview — รู้สถานการณ์ก่อนลงมือ

ผู้ใช้อย่างไรก็ไม่ควรทำอะไร หน้านี้ตอบคำถาม “มีอะไรต้องจัดการตอนนี้” และชี้ทางไป Flow ถัดไป



1 · ผู้ใช้มองหา

งานรอรับเรื่อง · งานในมือช่าง · งานเกินกำหนด

2 · สิ่งที่เกิดต่อ

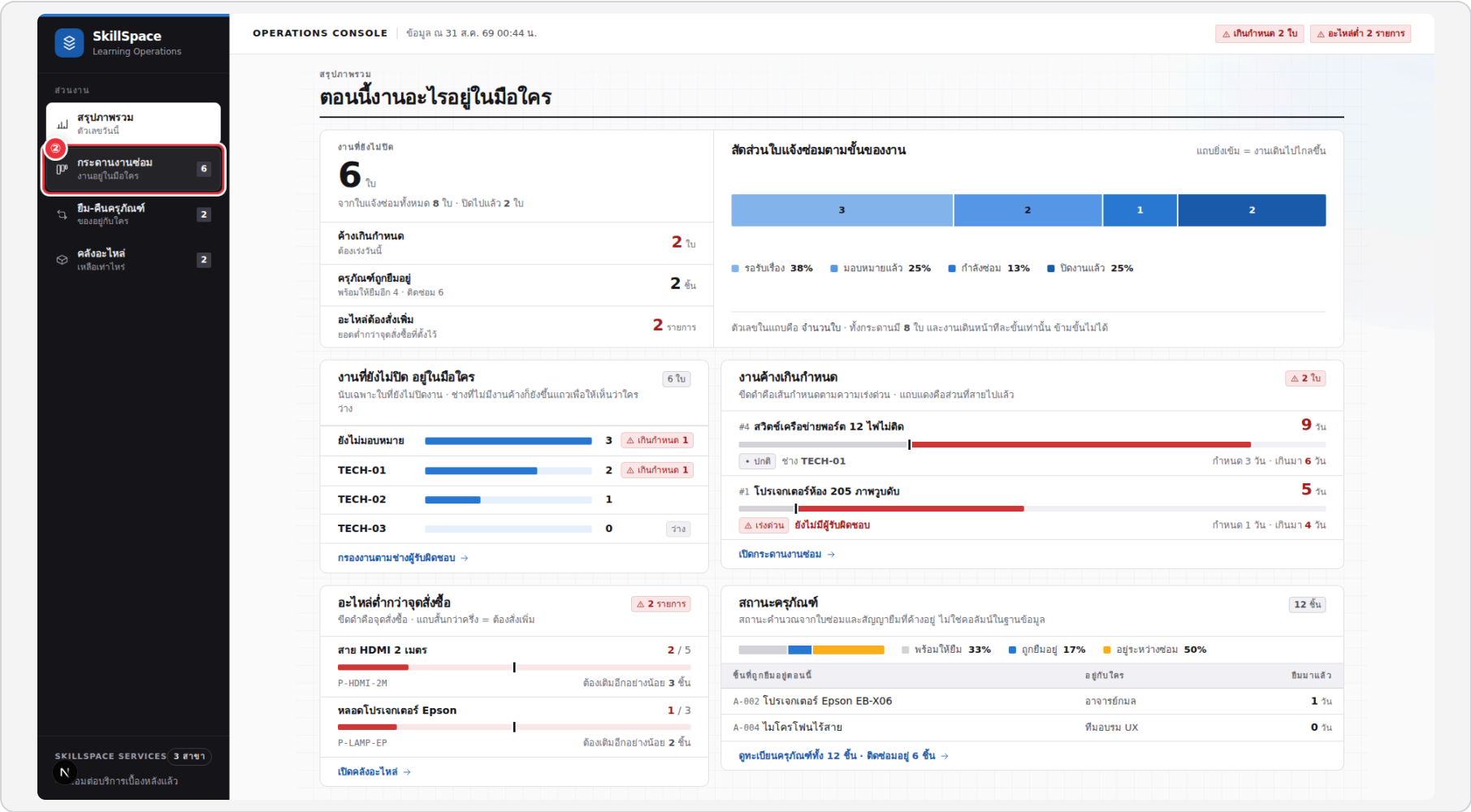
เมนู “กระดานงานซ่อม” เพื่อสร้างหรือติดตาม ticket

3 · ที่มาและหลักฐาน

US-6 → FR-08/09 · US-2 → FR-03 · US-9 → FR-12
ก่อน: งานไม่ปิด 6 · มอบหมาย 2 · ไม่มี TECH-04 → หลัง Flow ต้องเป็น 7 · 3 · TECH-04 = 1

เปิด Tickets — เห็นจุดเริ่มและปลายทางในหน้าเดียว

ด้านบนคือฟอร์มสร้างงาน · ด้านล่างคือกระดาน 4 State ที่บอกงานแต่ละใบอยู่ขั้นใด



1 · Input zone

ครุภัณฑ์ · หัวข้อ · รายละเอียด · ความเร่งด่วน

2 · Work zone

NEW → ASSIGNED → IN_PROGRESS
→ DONE

3 · ที่มาของหน้านี้

US-1 → FR-01 ให้สร้าง ticket · US-4 →
FR-02 ให้เดิน State ที่ละขั้น · US-2 →
FR-03 ให้มอบหมายช่าง

แยก "ของขึ้นไหน" · "เกิดอะไรขึ้น" · "เร่งแค่ไหน" เพื่อให้ข้อมูลพร้อมใช้ต่อ ไม่ใช่แค่ข้อความก่อกวนเดียว

SkillSpace
Learning Operations

ส่วนงาน

รูปภาพรวม

ตัวเลขวันนี้

กระดานงานซ่อม

งานอยู่ในมือใคร

6

ทีม-คืนครู/เกณฑ์

ของอยู่กับใคร

2

คลังอะไหล่

เหลือเท่าไร

2

SKILLSPACE SERVICES

3 สาขา

สนับสนุนบริการเบื้องต้น

ช่วยเหลือ

OPERATIONS CONSOLE

ข้อมูล ณ 31 ส.ค. 69 00:44 น.

เกินกำหนด 2 ปี

อะไหล่เก่า 2 รายการ

กระดานงานซ่อม

งานเดินที่ละชิ้น ข้ามชิ้นไม่ได้

กรองตามช่างผู้รับผิดชอบ

เช่น TECH-01

กรอง

แจ้งซ่อมใหม่

ใบใหม่เข้าคอลัมน์ "รอรับเรื่อง" เสมอ — จะข้ามไปมอบหมายทันทีไม่ได้

ครู/เกณฑ์

A-003 · กล้อง Sony ZV-1

หัวข้อ

กล้องถ่วงวิดีโอเปิดไม่ติด

รายละเอียดการ

กดปุ่มเปิดแล้วไฟสถานะไม่ทำงาน

ช่างเร่งด่วน

เร่งด่วน

แจ้งซ่อม

รอรับเรื่อง

รอช่างนำมมอบหมาย

3

ค้าง 5 วัน · กำหนด 1 วัน

เร่งด่วน

#1

โปรเจกเตอร์ห้อง 205 ภาพดับ

A-005 · โปรเจกเตอร์ BenQ MW550

เปิดไปสักพักภาพดับเอง ต้องถอดปลั๊กแล้วเสียบใหม่

ยังไม่มอบหมาย · ห้องเรียน 205

24 ส.ค. 69

ชื่อช่าง

มอบหมาย

ปกติ

#2

แอร์ห้องแล็บ 2 ไม่เย็น

A-006 · เครื่องปรับอากาศ Daikin

ลมออกแต่ไม่เย็น อาจต้องเติมน้ำยา

ยังไม่มอบหมาย · ห้องแล็บ 2

29 ส.ค. 69

ชื่อช่าง

มอบหมาย

ไม่เร่ง

#3

ลำโพงห้อง 402 ไม่ดีเสียงเบา

A-012 · ลำโพงห้อง 402

ระดับเสียงตกเป็นช่วง

ยังไม่มอบหมาย · ห้องเรียน 402

30 ส.ค. 69

ชื่อช่าง

มอบหมาย

มอบหมายแล้ว

ช่างรับแล้ว ยังไม่เริ่ม

2

ค้าง 9 วัน · กำหนด 3 วัน

ปกติ

#4

สวิตช์เครือข่ายพอร์ต 12 ไฟไม่ติด

A-008 · สวิตช์เครือข่าย 24 พอร์ต

เสียบสายแล้วไม่มีลิงก์ ย้ายไปพอร์ตอื่นใช้ได้

TECH-01 · ห้องเซิร์ฟเวอร์

20 ส.ค. 69

เริ่มลงมือซ่อม

เร่งด่วน

#5

โน้ตบุ๊กแล็บ 4 เปิดไม่ติด

A-009 · โน้ตบุ๊ก Lenovo ThinkPad

กดปุ่มแล้วไฟไม่ขึ้นเลย

TECH-02 · ห้องแล็บคอมพิวเตอร์

28 ส.ค. 69

เริ่มลงมือซ่อม

กำลังซ่อม

ช่างกำลังลงมือ

1

ปกติ

#6

เส้นสีชมพูขึ้นบนจอ

A-010 · จอแสดงผล LG 27 นิ้ว

เส้นสีชมพูกลางจอ

TECH-01 · ห้องแล็บคอมพิวเตอร์

28 ส.ค. 69

ปิดงาน + ปิดทีละโหล่ง >

ปิดงานแล้ว

ปิดพร้อมดีดอะไหล่แล้ว

2

ไม่เร่ง

#7

สแกนเนอร์สแกนแล้วมีเส้นดำ

A-011 · เครื่องพิมพ์ Canon

ทำความสะอาดกระจกแล้ว

TECH-03 · สำนักงาน ชั้น 2

17 ส.ค. 69

ปิดงานเมื่อ 20 ส.ค. 69

เร่งด่วน

#8

ลำโพงห้อง 402 ไม่มีเสียง

A-012 · ลำโพงห้อง 402

สายชำรุด เปลี่ยนสายใหม่

TECH-01 · ห้องเรียน 402

22 ส.ค. 69

ปิดงานเมื่อ 23 ส.ค. 69

US-1 → FR-01: ช่องเหล่านี้คือข้อมูลที่
ทำให้ “เรื่องไม่หาย” และข้างเริ่มงานได้โดยไม่
ต้องถามซ้ำ

กดแจ้งซ่อม — Success ต้องมองเห็นและตรวจได้

ผู้ใช้ไม่ควรต้องเดาว่าระบบรับเรื่องหรือยัง: หลังส่งสำเร็จต้องเห็นการ์ดใหม่ ตำแหน่งใหม่ และจำนวนใหม่

BEFORE

กด “แจ้งซ่อม”

Action เดียวที่ยืนยันข้อมูลทั้งฟอร์ม

AFTER

การ์ดใหม่ปรากฏ

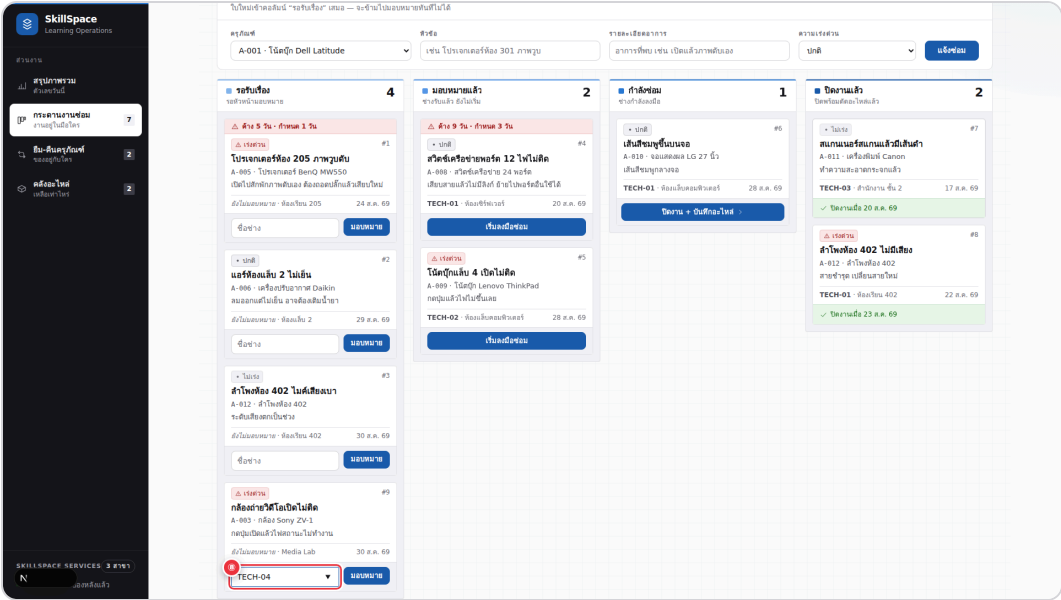
อยู่คอลัมน์ “รอรับเรื่อง” · จำนวนเพิ่ม · สถานะเริ่มต้น NEW

Visible success criteria · US-1 → FR-01: API 201 · เห็นการ์ดใหม่สถานะ NEW พร้อมหมายเลขอ้างอิง · ตัวเลขหัวคอลัมน์ 3 → 4 เป็นหลักฐานสนับสนุน FR-08

มอบหมายงาน — การ์ดเดิมเปลี่ยน State โดยไม่สร้างงานซ้ำ

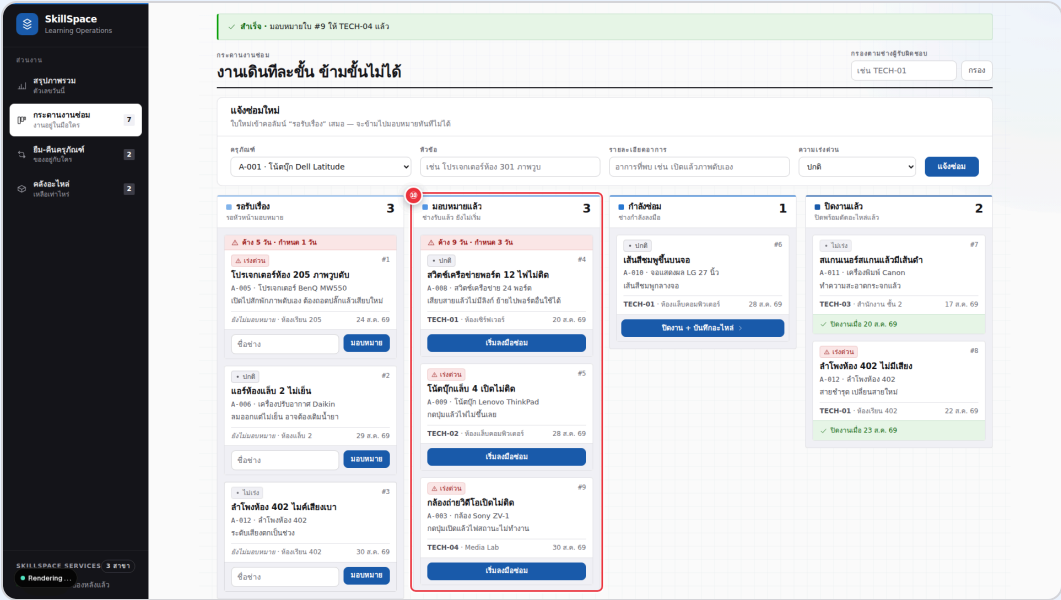
ผู้ใช้เลือกผู้รับผิดชอบบนการ์ดเดิม แล้วตรวจทั้ง “ชื่อช่าง” และ “คอลัมน์ที่ย้ายไป”

BEFOREเลือก Assignee



ชื่อช่างถูกผูกกับ ticket ที่เลือก ไม่ใช่ค่ารวมของทั้งหน้า

AFTERย้ายไป “มอบหมายแล้ว”



NEW → ASSIGNED · จำนวนสองคอลัมน์เปลี่ยนพร้อมกัน

หลักฐาน 3 จุด · US-2 → FR-03 · US-4 → FR-02: การ์ดยังเป็นใบเดิม · แสดงชื่อช่าง · อยู่คอลัมน์ใหม่ — ถ้าขาดข้อใดข้อหนึ่ง ผู้ใช้ยังยืนยันการมอบหมายไม่ได้

Flow อื่น ใช้หลักเดียวกัน — ผู้ใช้ต้องเห็นเหตุผลของผลลัพธ์

ไม่ว่าจะสำเร็จหรือถูกปฏิเสธ UI ต้องบอกว่าเกิดอะไรขึ้นและผู้ใช้ควรทำอะไรต่อ

/loans · US-7 → FR-10

เลือกของ → กดยืม → เห็น ASSET_ON_LOAN พร้อมว่าอยู่กับใคร · ภูคู้กัน US-8 → FR-11 ปฏิเสธ ASSET_IN_REPAIR (ไม่ได้แสดงในภาพนี้)

/parts · US-9 → FR-12

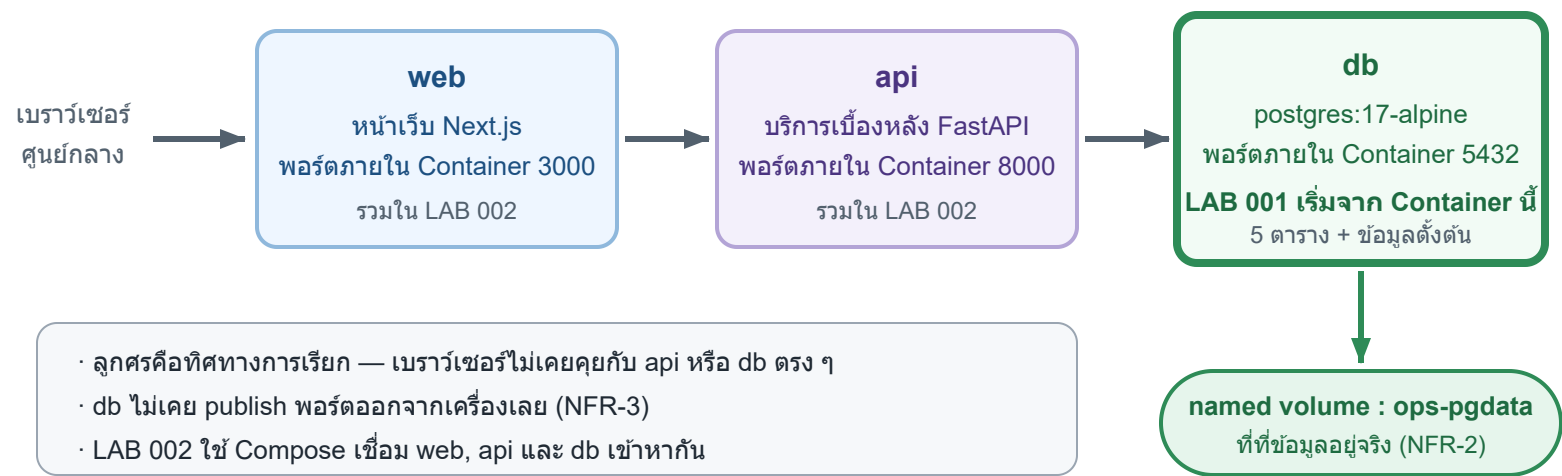
เปิดคลัง → เทียบยอด → เห็นสองรายการ “ต้องสั่งเพิ่ม” พร้อมค่าที่ใช้ตัดสินใจ

FLOW	ACTION	VISIBLE RESULT	ผู้ใช้ทำต่อ
ยืม-คืน	ขอยืมของที่ยังไม่มีคืน	ปฏิเสธ + เหตุผลเฉพาะ	เลือกของชิ้นอื่น/รอคืน
คลังอะไหล่	อ่านยอดคงเหลือ	ป้ายต่ำกว่าจุดสั่งซื้อ	เริ่มจัดซื้อ

3 Container : web · api · db

Architecture คือโครงสร้างและความสัมพันธ์ของส่วนประกอบ · Container แต่ละตัวรัน 1 service และสื่อสารกันผ่านเครือข่าย Docker

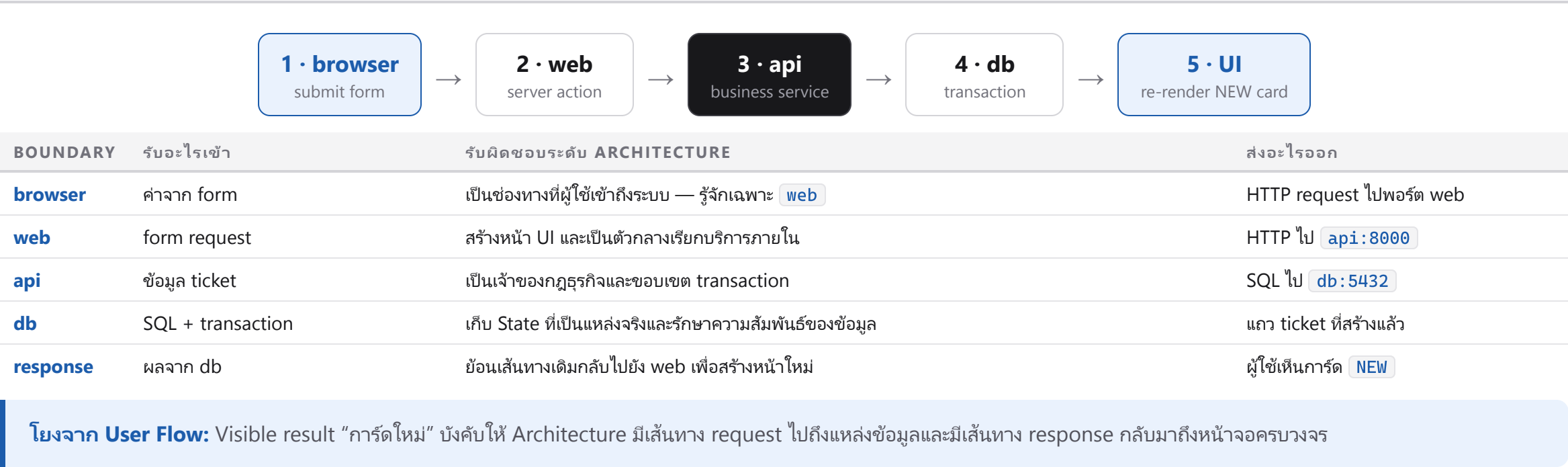
ระบบ SkillSpace = 3 Container · LAB 001 เตรียม db · LAB 002 ประกอบทั้งระบบ



SERVICE	IMAGE ที่ใช้	พอร์ตภายใน CONTAINER	เข้าถึงจากภายนอกเครื่อง
web	Next.js 16.3.1 (Dockerfile stage เดียว)	3000	เปิด ตามพอร์ตของแต่ละแล็บ — Container เดียวที่มี -p
api	FastAPI + uvicorn (build เอง)	8000	ปิด อยู่ใน network ภายในของ LAB 002
db	postgres:17-alpine (ใช้สำเร็จรูป)	5432	ปิด ไม่เคย publish เลยสักแล็บเดียว — NFR-3

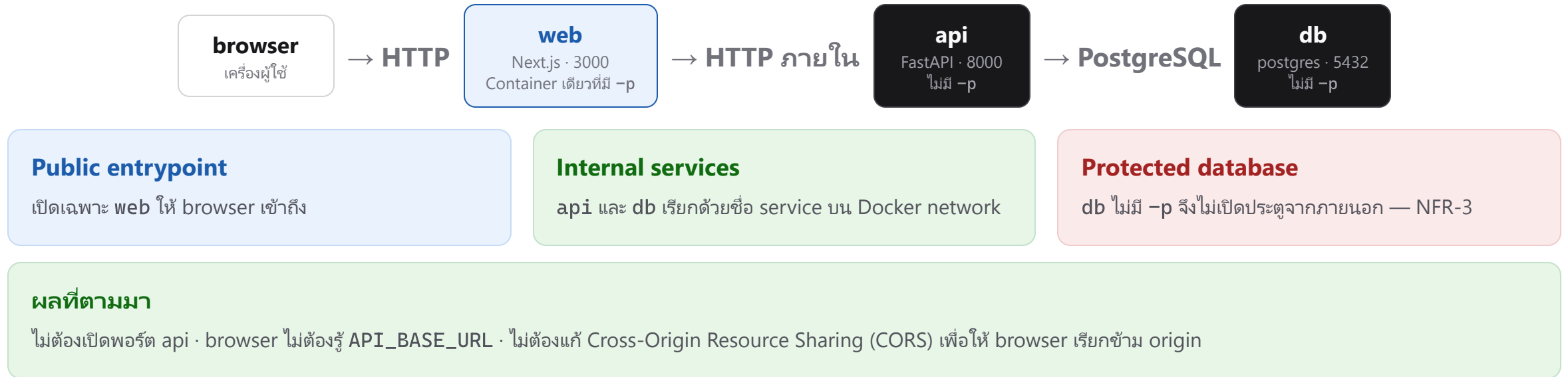
หนึ่งคลิกบน UI — ผ่านขอบเขตใดบ้างก่อนเห็นการ์ดใหม่

นำ Step 04 “กดแจ้งซ่อม” จาก User Flow มาแจกแจงเป็น Component และทิศทางข้อมูล โดยยังไม่ลงรายละเอียดโค้ด



เบราว์เซอร์ **ไม่เคย** เรียก api ตรง ๆ

จากมุมมองผู้ใช้มีเพียง URL ของ web; api และ db อยู่หลังขอบเขตเครือข่ายภายใน



แต่ละ Component รู้จัก ปลายทางถัดไป ผ่านค่าใด

ชื่อ service เป็น contract ระหว่าง Container; ไม่มี Component ใดต้องจำ Internet Protocol address (IP) ที่เปลี่ยนได้

ตัวแปรสภาพแวดล้อม	อ่านโดย	ค่าที่ใช้จริง	หน้าที่ใน ARCHITECTURE
POSTGRES_DB / USER / PASSWORD	db	skillspace / opsuser / labpass	กำหนด database และบัญชีภายใน
DATABASE_URL	api	postgresql://opsuser:labpass@db:5432/skillspace	api หา db ด้วยชื่อ db
API_BASE_URL	web server	http://api:8000	web หา api ด้วยชื่อ api
NEXT_PUBLIC_SITE_NAME	web build	SkillSpace	ชื่อที่ผู้ใช้เห็น ไม่ใช่ที่อยู่ service

Server-rendered UI

browser ขอหน้าจาก web; Next.js ขอข้อมูลจาก api ภายใน network แล้วสร้าง HTML ส่งกลับผู้ใช้

Dynamic data

force-dynamic ทำให้น่านอ่าน State ล่าสุดหลังผู้ใช้ submit หรือเปลี่ยนสถานะ ไม่ใช่ข้อมูลที่ฝังตอน build

จบ Architecture แล้ว: ตอนนี้รู้ว่า Component ใดรับผิดชอบอะไร เชื่อมไปทางไหน และขอบเขตใดเปิดต่อผู้ใช้ — หน้าถัดไปจึงค่อยตัดสินใจรายละเอียดภายในแต่ละ Component

ทุกเส้นเชื่อมต่อต้องตอบได้ — ส่งอะไร ด้วยอะไร และล้มแล้วเกิดอะไร

กล่องอย่างเดียวน่าพอ: Architecture ที่นำไปสร้างและตรวจได้ต้องอธิบาย protocol, data, direction และ failure behavior

เส้นเชื่อม	PROTOCOL / INTERFACE	ข้อมูลหรือคำสั่ง	เมื่อปลายทางผิดพลาด	สิ่งที่ผู้ใช้เห็น
browser → web	HTTP บน published port	page request · form fields	เชื่อมต่อไม่ได้หรือ web ไม่พร้อม	หน้าโหลดไม่สำเร็จ — ไม่อ้างว่าสร้างงานแล้ว
web → api	HTTP + JSON ภายใน network	POST /api/tickets · PATCH /api/tickets/{id}/status	timeout / 4xx / 5xx; web แยก error ที่แก้ข้อมูลได้จาก service error	ข้อความเฉพาะ เช่น ต้องระบุชื่อช่าง หรือข้อความให้ลองใหม่
api → db	PostgreSQL protocol	INSERT / UPDATE / SELECT ใน transaction	rollback เมื่อ query หรือ constraint ไม่ผ่าน; ห้ามตอบ success ล่วงหน้า	ไม่มีการดหรือ State ใหม่จน commit สำเร็จ
db → volume	filesystem mount	PostgreSQL data directory	Container สร้างใหม่ได้โดยข้อมูลยังอยู่; ลบ volume จึงสูญข้อมูล	ข้อมูลเดิมกลับมาหลัง restart/down-up — NFR-2

Direction

request เดินซ้าย → ขวา; response ย้อนกลับขวา → ซ้าย

Atomic result

ตอบสำเร็จหลัง transaction commit เท่านั้น

No false success

ถ้าปลายทางล้ม UI ต้องไม่สร้าง State ที่ database ไม่มี

ขอบเขตของชุดนี้: ไม่มี Gateway, Broker, External Service หรือ LLM เพราะ User Flow และ NFR ไม่ได้บังคับให้มี — การไม่เพิ่ม Component ที่ไม่จำเป็นคือ Design decision เช่น กัน

จากช่องที่ผู้ใช้กรอก — กลายเป็น Contract อะไร

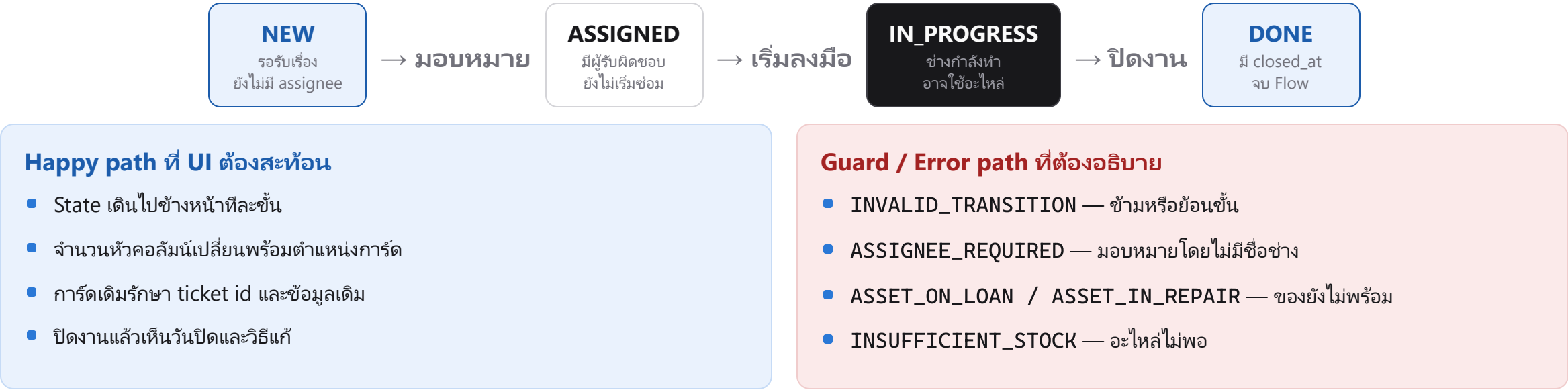
System Design แปลงทุก Visible State ใน User Flow ให้เป็น field, validation, endpoint, data และ response ที่สร้างได้จริง

สิ่งที่ผู้ใช้เห็น	FIELD / VALIDATION	API CONTRACT	DATA	ผลกลับสู่ UI
เลือกครุภัณฑ์	<code>asset_id</code> ต้องมีจริง	<code>POST /api/tickets</code>	<code>tickets.asset_id</code> → <code>assets.id</code>	แสดงรหัสและชื่อของชิ้นเดิม
หัวข้อ	<code>title</code> ห้ามว่าง	สำเร็จ <code>201</code> ข้อมูลไม่ครบ <code>422</code>	<code>tickets.title</code>	หัวข้อบนการ์ด
รายละเอียด	<code>detail</code> เป็นข้อความ		<code>tickets.detail</code>	บริบทสำหรับช่าง
ความเร่งด่วน	<code>LOW NORMAL HIGH</code>		<code>tickets.priority</code>	ป้ายระดับ + การคำนวณ SLA
กดแจ้งซ่อม	สร้างด้วย State เริ่มต้น	response มี ticket id	<code>status = NEW</code>	การ์ดอยู่ "รอรับเรื่อง"
กดมอบหมาย	<code>assignee</code> ห้ามว่าง	<code>PATCH /api/tickets/{id}/status</code>	<code>NEW → ASSIGNED</code>	การ์ดย้ายคอลัมน์ + แสดงชื่อช่าง

Design review rule: ทุก field ต้องมีที่มาใน UI · ทุก response ต้องสร้าง Visible State · ทุก State ต้องเก็บหรือคำนวณจากข้อมูลจริง

กระดาน 4 คอลัมน์ — ไม่ใช่แค่ Layout แต่คือกฎ State

UI แสดงตำแหน่งของงาน ส่วน API เป็นผู้อนุญาตการเปลี่ยนขั้น และ database เก็บ State ล่าสุด



หลักออกแบบ: Disable ปุ่มช่วยป้องกันความผิดพลาดบน UI แต่ API ต้องตรวจสอบ เพราะ request อาจไม่ได้มาจากหน้าจอ

System Design ต้องอธิบายครบ — ไม่มี “กล่องลอย ๆ”

ทุกส่วนระบุ Technology, Input/Output, หน้าที่, การเชื่อมต่อ และเหตุผลว่าทำไมอยู่ในระบบหรือทำไมตั้งใจไม่ใช้

ส่วนที่ต้องอธิบาย	สิ่งที่เลือกใช้	หน้าที่และข้อมูลที่ได้รับผิดชอบ	ขอบเขต / วิธีตรวจ
User / Actor	ผู้จัดการ · ช่าง · พัสตุ	แจ้ง/มอบหมาย/ซ่อม · ยืมคืน · ดูแลสต็อก	ไม่มี login/role ตามขอบเขตที่ลูกค้าระบุ; ใครเปิด UI ได้ใช้ทุกฟังก์ชัน
Frontend / Interface	Next.js 16 · server-rendered	รับ form input · แสดง board/dashboard/error · เรียก API จาก server side	ตรวจจาก 4 หน้าและ Visible State ก่อน/หลัง action
Backend / API	FastAPI + Python	validation · transition · loan/repair/stock rules · transaction boundary	ตรวจ endpoint ด้วย Swagger/curl และ HTTP 201/400/404/409/422
Database / Storage	PostgreSQL 17 + named volume	5 tables · foreign key · check constraint · persistence	schema ก่อน seed; ตรวจ row จริงและ down เทียบ down -v
Integration	Docker network + service DNS	web → api → db ด้วยชื่อ service; เปิดภายนอกเฉพาะ web	ไม่มี Gateway/Broker/Cache/External Service เพราะ Flow ปัจจุบันไม่ต้องใช้
LLM / AI Module	ไม่ใช่	กฎทั้งหมด deterministic และตรวจรับได้จาก input/output ที่กำหนด	ไม่เพิ่ม prompt, provider, secret หรือ generated output ที่ Requirement ไม่ได้ขอ
Deployment / Verification	Dockerfile + Compose	3 images/services · healthcheck · dependency order · restart policy	docker compose up -d --build + smoke/verify script + UI evidence

Design decision ต้องตรวจย้อนกลับได้: ถ้าส่วนใดไม่มี User Flow, Requirement หรือ NFR รองรับ ต้องอธิบายเหตุผลก่อนเพิ่ม; ถ้ารับปากในสไลด์ ต้องมี Implementation และ Evidence จริง

ข้อมูล 5 กลุ่ม — เชื่อมกันอย่างไร

มองความสัมพันธ์ก่อนอ่าน SQL: ข้อมูลแต่ละกลุ่มเกิดจากงานของผู้ใช้ และช่วยพิสูจน์ Requirement คนละส่วน

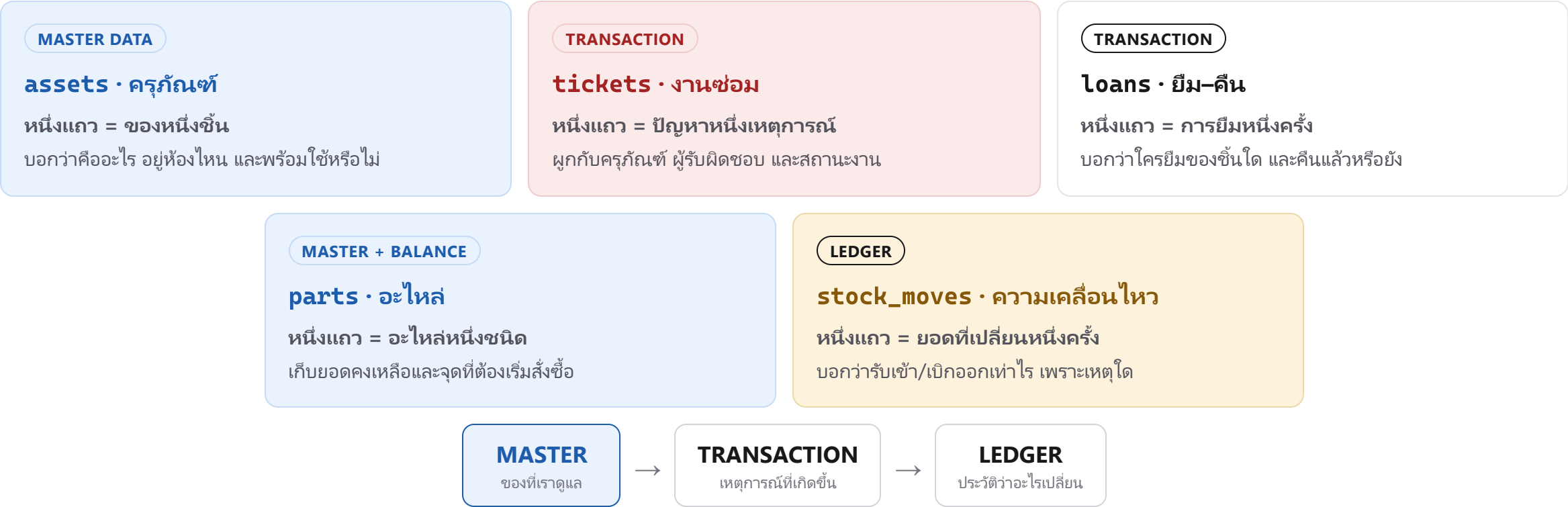


อ่านภาพนี้ก่อน: assets เชื่อมงานซ่อมและการยืม · tickets เชื่อมการเบิกอะไหล่ · stock_moves เป็นประวัติที่อธิบายว่าทำไมยอด parts จึงเปลี่ยน

ข้อควรระวัง กฎ UNIQUE/CHECK และ SQL เต็มจะค่อยเปิดใน LAB 001 เมื่อเริ่มสร้าง database จริง

แต่ละ Table รับผิดชอบข้อมูลอะไร?

Table ไม่ใช่แค่กล่องเก็บข้อมูล แต่เป็น “เจ้าของความจริง” คนละประเภท · โครงสร้างทั้ง 5 ตารางอยู่ใน 01-schema.sql



จำประโยคเดียว: assets/parts บอกว่า “เรามีอะไร” · tickets/loans บอกว่า “เกิดอะไรขึ้น” · stock_moves บอกว่า “ยอดเปลี่ยนเพราะอะไร”

แหล่งที่มา: 01-schema.sql สร้างโครงทั้ง 5 ตาราง · 02-seed.sql เติมแถวตัวอย่างให้ทั้ง 5 ตาราง

assets — หนึ่งแถวคือครุภัณฑ์หนึ่งชิ้น

ตัวอย่าง 6 จาก 12 แถวจริง · code ใช้อ้างอิงของชิ้นเดิมข้ามงานซ่อมและการยืม

CODE	NAME	LOCATION	นำไปเชื่อมกับ
A-001	โน้ตบุ๊ก Dell Latitude	ห้องพัสดุ ชั้น 1	loans
A-002	โปรเจกเตอร์ Epson EB-X51	ห้องอบรม 301	loans
A-003	กล้อง Sony ZV-1	ห้องพัสดุ ชั้น 1	loans
A-004	ไมโครโฟนไร้สาย Shure	ห้องประชุมใหญ่	พร้อมใช้งาน
A-005	โปรเจกเตอร์ BenQ MW550	ห้องอบรม 205	tickets
A-006	เครื่องปรับอากาศ ห้องแล็บ 2	ห้องแล็บคอมพิวเตอร์ 2	tickets

เชื่อมกับ Data Model หน้า 23: assets.id คือ Parent Key ที่ tickets.asset_id และ loans.asset_id ใช้อ้างอิง

tickets — หนึ่งแถวคือปัญหาหนึ่งเหตุการณ์

ตัวอย่าง 6 จาก 8 แถวจริง · งานของครุภัณฑ์คนละชิ้นจึงมีสถานะ ความเร่งด่วน และผู้รับผิดชอบต่างกัน

ASSET	TITLE	PRIORITY	STATUS	ASSIGNEE
A-005	โปรเจกเตอร์ห้อง 205 ภาพวูบดับ	HIGH	NEW	—
A-006	แอร์ห้องแล็บ 2 ไม่เย็น	NORMAL	NEW	—
A-007	เครื่องพิมพ์ป้อนกระดาษช้อน	LOW	NEW	—
A-008	สวิตช์เครือข่ายพอร์ต 12 ไฟไม่ติด	NORMAL	ASSIGNED	TECH-01
A-009	โน้ตบุ๊กแล็บ 4 เปิดไม่ติด	HIGH	ASSIGNED	TECH-02
A-010	จอแล็บ 1 มีเส้นแนวตั้ง	NORMAL	IN_PROGRESS	TECH-01

Requirement ที่มองเห็นจากข้อมูล: FR-02 ใช้ status · FR-03/04 ใช้ assignee · FR-09 ใช้ priority + created_at

Loans — หนึ่งแถวคือการยืมหนึ่งครั้ง

มีจริง 3 แถวและแสดงครบทั้งหมด · ตารางนี้ตอบว่า “ของชิ้นใดถูกยืมโดยใคร และคืนแล้วหรือยัง”

ASSET	BORROWER	BORROWED_AT	RETURNED_AT
A-001	วิทยากรหลักสูตร Data 101	4 วันที่แล้ว	NULL · ยังไม่คืน
A-002	เจ้าหน้าที่ธุรการ	1 วันที่แล้ว	NULL · ยังไม่คืน
A-003	วิทยากรหลักสูตร UX 204	10 วันที่แล้ว	คืนแล้วเมื่อ 6 วันที่แล้ว

FR-10 · ห้ามยืมซ้ำ

ถ้ามีแถวของครุภัณฑ์ชิ้นนั้นที่ `returned_at` = NULL ระบบต้องปฏิเสธการยืมใหม่

FR-11 · ห้ามยืมของที่กำลังซ่อม

ตรวจ `tickets` เพิ่มว่าอุปกรณ์ชิ้นเดียวกันยังมียางานที่ไม่เป็น DONE หรือไม่

จบกลุ่ม A: `assets` คือของ · `tickets` คืองานซ่อม · `loans` คือเหตุการณ์ยืม-คืน

parts — ยอดคงเหลือปัจจุบันของอะไหล่

มีจริง 6 แถวและแสดงครบทั้งหมด · ใช้ “จำนวนคงเหลือ” เทียบกับ “จุดสั่งซื้อขั้นต่ำ” เพื่อสร้างรายการเตือน

จำนวนคงเหลือในคลัง
qty_on_hand · จำนวนอะไหล่ที่มีอยู่จริงและพร้อมเบิกในขณะนี้

จุดสั่งซื้อขั้นต่ำ
reorder_point · ระดับที่กำหนดไว้เพื่อเตือนให้เริ่มสั่งของเพิ่มก่อนหมด

รหัสอะไหล่	ชื่ออะไหล่	จำนวนคงเหลือ	จุดสั่งซื้อขั้นต่ำ	ผล
LAMP-EPS-01	หลอดโปรเจกเตอร์ Epson	2	5	ต้องสั่งเพิ่ม
FAN-NB-14	พัดลมระบายความร้อนโน้ตบุ๊ก	12	4	เพียงพอ
KBD-USB-01	คีย์บอร์ด USB	8	3	เพียงพอ
CBL-HDMI-3M	สาย HDMI ยาว 3 เมตร	1	6	ต้องสั่งเพิ่ม
SSD-256	SSD 256GB	20	5	เพียงพอ
MIC-CAP-01	หัวไมโครโฟนสำรอง	9	2	เพียงพอ

กติกาของ FR-12: เมื่อ จำนวนคงเหลือ < จุดสั่งซื้อขั้นต่ำ ระบบต้องเตือนให้สั่งเพิ่ม — ชุดข้อมูลนี้พบสองชนิด

stock_moves — ครบ 6 แถว อธิบายทุกครั้งที่ยอดเปลี่ยน

หนึ่งแถวคือเหตุการณ์ที่ทำให้จำนวนอะไหล่เปลี่ยน พร้อมบอกจำนวน เหตุผล และงานซ่อมต้นทางเมื่อเกี่ยวข้อง

จำนวนที่เปลี่ยน · delta
ค่าบวกคือรับเข้า · ค่าติดลบคือเบิกออก

เหตุผล · reason
อธิบายว่าทำไมยอดจึงเพิ่มหรือลด

งานซ่อมที่เกี่ยวข้อง · ticket_id
ชี้กลับไปยังใบงานที่เบิกอะไหล่; ชัดหมายถึงไม่ผูกงานซ่อม

อะไหล่ · PART	จำนวนที่เปลี่ยน · DELTA	เหตุผล · REASON	งานซ่อมที่เกี่ยวข้อง · TICKET_ID
LAMP-EPS-01	+5	รับเข้าจากผู้ขาย	—
LAMP-EPS-01	-3	เบิกใช้เตรียมห้องอบรมประจำเดือน	—
CBL-HDMI-3M	+6	รับเข้าจากผู้ขาย	—
CBL-HDMI-3M	-5	ใช้ในงานซ่อม	ลำโพงห้องอบรม 402
KBD-USB-01	+9	รับเข้าจากผู้ขาย	—
KBD-USB-01	-1	ใช้ในงานซ่อม	เครื่องสแกนมีเส้นดำ

อ่านตัวอย่างแถวที่ 4: delta = -5 หมายถึงเบิกสาย HDMI ออก 5 เส้น เพราะใช้กับงานซ่อมลำโพงห้องอบรม 402

assets + tickets — รองรับ Requirement ใดบ้าง?

assets บอกว่า “ของชิ้นไหน” · tickets บอกว่า “ปัญหาอะไร อยู่ชั้นไหน และใครรับผิดชอบ”

REQ	ความสามารถที่ผู้ใช้ต้องได้	ข้อมูลที่ใช้	TABLE
FR-01	สร้างใบแจ้งปัญหาโดยระบุครุภัณฑ์ อาการ และความเร่งด่วน	asset_id · title · priority	assets → tickets
FR-02	เปลี่ยนสถานะงานตามลำดับที่กำหนด	tickets.status	tickets
FR-03	มอบหมายงานพร้อมชื่อช่างผู้รับผิดชอบ	tickets.assignee	tickets
FR-04	กรองรายการตามช่างผู้รับผิดชอบ	assignee + index สำหรับค้นหา	tickets
FR-08	สรุปจำนวนงานแยกตามสถานะ	นับแถวตาม status	tickets
FR-09	ชิ้นงานที่ค้างเกินกำหนดตามระดับความเร่งด่วน	priority · created_at · status	tickets

ความสัมพันธ์: ครุภัณฑ์หนึ่งชิ้นใน assets มีใบแจ้งซ่อมได้หลายใบใน tickets ตลอดอายุการใช้งาน

loans + parts + stock_moves — แต่ละ REQ คืออะไร?

แยก “รายการยืม” ออกจาก “ยอดอะไหล่” และใช้ Ledger อธิบายทุกครั้งที่ยอดสต็อกเปลี่ยน

REQ	ความสามารถที่ผู้ใช้ต้องได้	ข้อมูลที่ใช้ตัดสิน	TABLE
FR-10	ห้ามยืมครุภัณฑ์ที่ยังไม่ถูกคืน	<code>returned_at IS NULL</code>	<code>loans</code>
FR-11	ห้ามยืมครุภัณฑ์ที่กำลังซ่อม	งานซ่อมของชิ้นนั้นยังไม่เป็น <code>DONE</code>	<code>tickets</code> + <code>loans</code>
FR-05	ปิดงานพร้อมบันทึกอะไหล่ที่ใช้	<code>ticket_id</code> · จำนวนที่เบิกออก	<code>tickets</code> + <code>stock_moves</code>
FR-06	ห้ามเบิกอะไหล่มากกว่ายอดที่มี	<code>qty_on_hand</code> ต้องไม่ติดลบ	<code>parts</code>
FR-07	เปิดดูประวัติรับเข้าและเบิกออก	<code>delta</code> · <code>reason</code> · <code>created_at</code>	<code>stock_moves</code>
FR-12	เตือนเมื่ออะไหล่ต่ำกว่าจุดสั่งซื้อ	<code>qty_on_hand < reorder_point</code>	<code>parts</code>

จำเป็นภาพ: loans ตอบว่า “ของถูกยืมอยู่ไหม” · parts ตอบว่า “เหลือเท่าไร” · stock_moves ตอบว่า “ยอดเปลี่ยนเพราะอะไร”

หนึ่ง database · ห้า tables · สองไฟล์ตามลำดับ

LAB 001 ให้ Python อ่านโครงสร้างจริงผ่าน information_schema หลัง PostgreSQL ประมวลผล schema และ seed

01-schema.sql

สร้าง assets, tickets, loans, parts, stock_moves ภายใน database
skillspace

02-seed.sql

เติมข้อมูลตัวอย่างให้ตารางทั้งห้า หลัง schema พร้อมแล้ว

```
db/initdb/
```

```
|— 01-schema.sql # CREATE TABLE ก่อน
```

```
|— 02-seed.sql # INSERT INTO ที่หลัง
```

คำที่ต้องใช้ให้ตรง: มีฐานข้อมูล PostgreSQL หนึ่งฐานชื่อ skillspace และมีห้าตารางภายในฐานนั้น

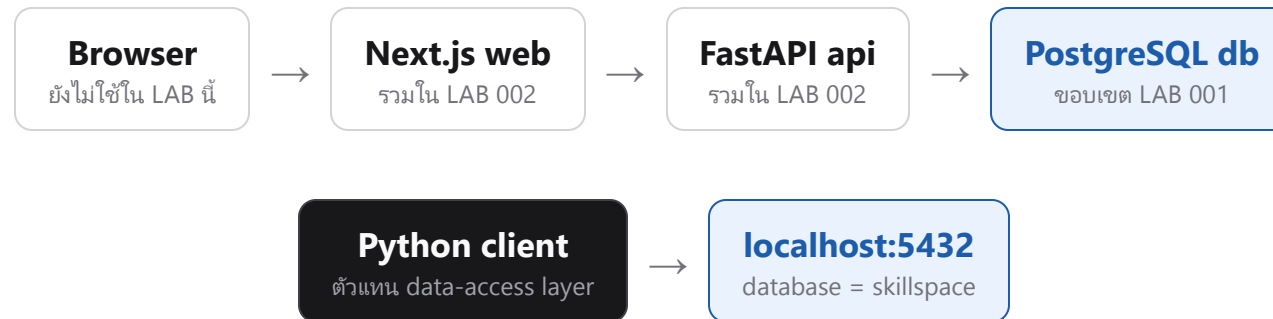
04

LAB 001 — PostgreSQL

หนึ่ง database *skillspace* · ห้า tables · schema + seed · Python client

เตรียมเครื่องปฏิบัติการ → สร้าง PostgreSQL → เชื่อมต่อ → สืบค้นข้อมูล → cleanup · 5 การทดลอง

เส้นทางข้อมูลของแล็บนี้ หยุดที่ PostgreSQL



จำให้แม่น: Browser ไม่ต่อฐานข้อมูลโดยตรง และแล็บนี้ไม่ใช่ docker exec หรือ psql จัดการข้อมูล

05

LAB 002 — Fullstack Compose

ประกอบ web · api · db เป็นระบบเดียว

เปิดระบบ → ทดลองงานซ่อม → ตรวจ Loans และ Parts → พิสูจน์ persistence · 10 การทดลอง

เปิดระบบหนึ่งคำสั่ง แล้วเดินงานซ่อมให้ครบหนึ่งรอบ

สไลด์ให้ภาพรวมเท่านั้น · ขั้นตอนพร้อมภาพผลลัพธ์อยู่ใน README ของ LAB 002

1 · เปิดระบบ

docker compose up -d --build

2 · ทดลอง User Flow

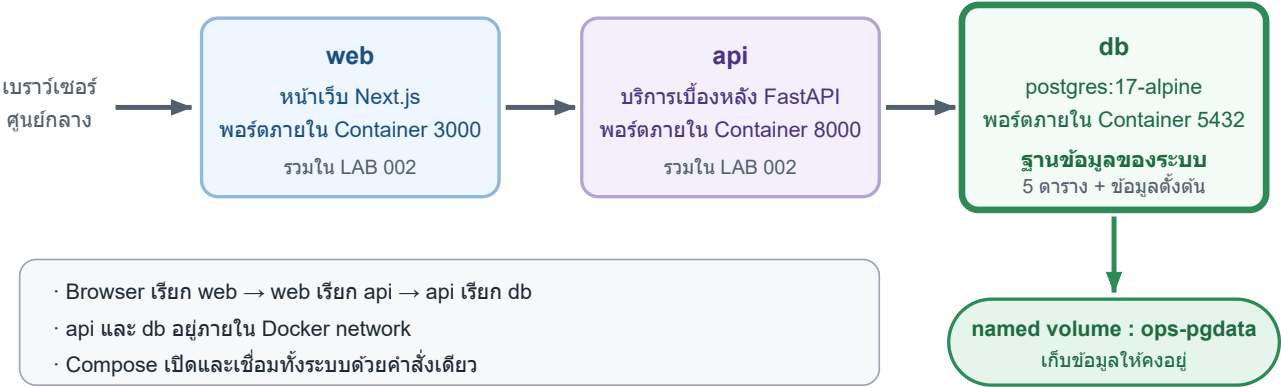
Overview → Tickets → Create → Assign → Repair → Close

3 · ตรวจสอบผล

Loans · Parts · Persistence · bash verify.sh

คู่มือปฏิบัติ: 002_LAB_Fullstack_Compose/readme.md

LAB 002 ประกอบระบบเต็มด้วย Docker Compose



06

LAB 003 — Lost Pet Finder

อ่าน Evidence และ Requirements แล้วออกแบบ User Flow กับ Wireframe

สร้างประกาศ → ชุมชนสิ่งเบาะแส → เจ้าของตรวจสอบ → อัปเดตสถานะพบแล้ว

Lost Pet Finder — ออกแบบระบบประกาศและติดตามสัตว์เลี้ยงหาย

อ่าน Evidence และ Requirements แล้วสร้างคำตอบของทีมเอง · User Flow + Wireframe

1 · เข้าใจ Pain Point

เจ้าของสัตว์ · ผู้พบเห็น · อาสาสมัคร · ศูนย์พักพิง

2 · ออกแบบ User Flow

สร้างประกาศ → รับเบาะแส → อัปเดตสถานะ

3 · สร้าง Wireframe

ส่งเป็นภาพถ่าย ภาพดิจิทัล/AI หรือไฟล์ HTML ก็ได้

โจทย์และสิ่งที่ต้องส่ง: 003_LAB_Lost_Pet_Finder/readme.md

